

RealityKit Asset and Texture Pipeline Guide

Blender / USDZ / Xcode / RealityKit öğrenme rehberi

RealityKitPipelineDemo

2026-05-06

RealityKit Asset and Texture Pipeline Guide

Bu rehber, Kyylia ve Mehmet'in mobil RealityKit oyunlarında asset ve texture pipeline'ını birlikte öğrenmesi için yazıldı. Amaç sadece bu demoyu çalıştırmak değil; bir `.usdz` asset'in fikir aşamasından simulator screenshot'ına kadar hangi adımlardan geçtiğini anlamak ve aynı süreci yeni assetlerde tekrar edebilmek.

Executive Summary

Bu proje üç şeyi aynı anda öğretir:

- Runtime:** SwiftUI + RealityKit sahnesi, input, target spawn, projectile ve collision.
- Asset pipeline:** Blender/üretim aracı -> USDZ -> Xcode resource bundle -> RealityKit loader.
- Production discipline:** Her asset işi manifest, build, screenshot ve öğrenme notuyla kapanır.

En önemli prensip: Kalıcı rol ayrımı yok. Bir kişi sadece Blender, diğer kişi sadece kod tarafını bilmeyecek. İş bölümü yapılabilir; fakat her handoff sonunda ikiniz de asset'in amacı, scale/origin kararı, UV/material davranışı, bundle yolu ve RealityKit doğrulamasını açıklayabilmelisiniz.

Guide System

Bu repo artık üç katmanlı bir rehber sistemi olarak düşünölmeli:

Dosya	Ne zaman kullanılır?
<code>Docs/guide.md</code>	Asset ve texture pipeline'ını öğrenirken.
<code>Docs/production-playbook.md</code>	Gerçek feature/asset işi planlarken, review ederken veya release hazırlarken.
<code>Docs/new-game-startup.md</code>	Bu repo'dan öğrenilen sistemi yeni bir RealityKit oyununa taşıırken.
<code>Skills/realitykit-pipeline-guide</code>	Aynı sistemi Codex skill olarak tekrar kullanırken.

Kural: Öğrenme sırasında önce bu dosyayı oku. Uygulama sırasında `production-playbook` kapısını kullan. Yeni proje başlatırken `new-game-startup` checklist'ini takip et.

Skill kurulumu:

```
make install-skill
```

Kurulumdan sonra Codex'e `Use realitykit-pipeline-guide` diyerek asset, gameplay, release veya dokümantasyon görevlerini bu repo standardına göre yönlendirebilirsiniz.

Scope Statement

Bu rehber şu anda **tam bir oyun geliştirme kursu değildir**. Mevcut kapsam, RealityKit tabanlı küçük bir playable prototype içinde asset ve texture pipeline'ını öğretmektir.

Şu anda güçlü kapsanan alan:

- USDZ asset import
- scale/origin/orientation doğrulama
- base color texture pipeline
- UV primvar debugging
- Xcode resource bundle

- RealityKit loader fallback
- manifest/worklog disiplini
- screenshot-based visual QA

Henüz tam kapsanmayan alan:

- game loop architecture
- wave/difficulty/scoring design
- input feel
- hit VFX, audio, haptics
- menu/tutorial/results UI
- persistence/settings
- device performance profiling
- release workflow

Bu nedenle repo'nun mevcut vaadi şudur:

Learn how a RealityKit game prototype loads, validates, documents, and teaches a mobile 3D asset/texture pipeline.

Tam oyun geliştirme eğitime dönüşmesi için [Game Development Curriculum Roadmap](#) bölümündeki modüller doldurulmalıdır.

How to Use This Guide

Hedef Kitle

- RealityKit öğrenen iOS/macOS geliştiricileri.
- Blender veya 3D asset workflow'una yeni giren ekip üyeleri.
- Oyun prototipinde asset pipeline'ı sistemli kurmak isteyen küçük ekipler.

Ön Koşullar

- Xcode kurulu.
- XcodeGen kurulu.
- Blender 4.x veya Blender MCP tabanlı asset üretim yolu mevcut.

- Temel terminal ve SwiftUI bilgisi.

Not: Worklog ve agent dosyalarında `rtk` prefix'i görebilirsiniz. `rtk` bu çalışma ortamındaki lokal ajan wrapper'ıdır, public dependency değildir. Repo'yu normal klonladıysanız aynı komutu `rtk` olmadan çalıştırın veya `Makefile` hedeflerini kullanın.

Önerilen Çalışma Formatı

Süre	Aktivite
10 dk	Bu rehberdeki mental model ve proje haritasını okuyun.
20 dk	Sprint 1-3 walkthrough'larını screenshot'larla inceleyin.
30 dk	Yeni asset checklist'ini takip ederek küçük bir varyasyon üretin.
15 dk	Debugging playbook ile sonucu yorumlayın.
10 dk	Worklog'a ne öğrendiğinizi yazın.

Completion Standard

Bir asset işi, ancak şu kanıtlarla kapanır:

- `Tools/asset_manifest.json` içinde doğru status ve not var.
- `rkp verify-asset <asset_id>` veya eşdeğer kalite kapıları geçiyor.
- `rkp inspect-usdz <asset_id>` texture varlığı, texture boyutu, bilinen triangle bütçesi ve text USD `st` UV sinyalinin raporluyor.
- Simulator screenshot alındı.
- HUD veya sahne asset'in gerçekten yüklendiğini gösteriyor.
- Öğrenme notu `Docs/WORKLOG.md` veya checklist'e işlendi.

CLI Quality Gate

RKP'nin güncel ürün yüzeyi sadece dosya scaffold etmek değildir; asset draft üretir ve acceptance öncesi kalite kapıları çalıştırır.

En kısa yeni asset akışı:

```
rkp init --project-name MyGame
rkp make-asset enemy_drone --type gameplay_target --prompt "red bullseye drone target"
rkp build-asset enemy_drone
rkp verify-asset enemy_drone
```

Tek komutta acceptance'a kadar gitmek için:

```
rkp verify-asset enemy_drone \
  --build \
  --screenshot Docs/screenshots/enemy_drone_imported.jpg \
  --release-check
```

Bu komut sırasıyla şunları yapar:

1. `--build` verilirse USDZ üretir.
2. `inspect-usdz` ile paketi kontrol eder.
3. Screenshot verilirse `accept-asset` ile asset'i kabul eder.
4. `--release-check` verilirse repo kalite kapısını çalıştırır.

`inspect-usdz` kabulden önceki otomatik paket denetimidir:

```
rkp inspect-usdz enemy_drone
rkp inspect-usdz enemy_drone --json
```

Kontrol ettiği şeyler:

- USDZ dosyası var mı ve boş değil mi?
- Beklenen base color texture paket içinde mi?
- PNG/JPEG texture boyutu manifest `maxTextureSize` değerini aşıyor mu?
- Text `.usda/.usd` içinde `primvars:st` UV sinyali var mı?
- Text USD'den okunabilen face count manifest `maxTriangles` bütçesini aşıyor mu?

Binary `.usdc` paketlerde geometry ve UV sayıları uydurulmaz; bilinmeyen alanlar `unknown` raporlanır. Bu durumda screenshot acceptance hâlâ son kanıttır.

Asset Draft Üretim Yolları

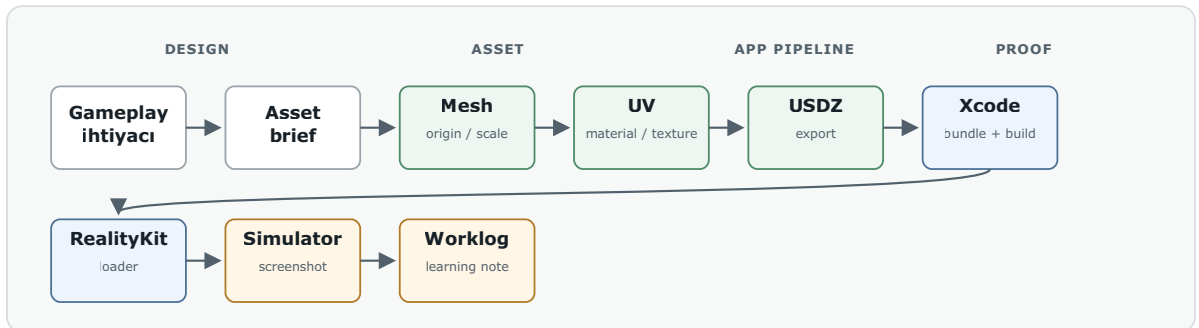
RKP birden fazla draft üretim yolu destekler:

Yol	Komut	Ne üretir?
Deterministic template		

Yol	Komut	Ne üretir?
	<code>rkp prompt-asset <id> --prompt "..."</code>	Manifest, brief ve procedural Blender generator
Blender build	<code>rkp build-asset <id></code>	Blender script'ten USDZ
Meshy backend	<code>rkp make-asset <id> --backend meshy --prompt "..."</code>	Meshy text-to-3D USDZ draft
Claude generator	<code>rkp prompt-asset <id> --generator claude --prompt "..."</code>	Claude-authored Blender script

Default generator deterministiktir. API key ortamda olsa bile Claude otomatik çağrılmaz; `--generator claude` açıkça verilmelidir. Meshy ve Claude çıktıları da final değil, draft kabul edilir. Production acceptance için yine `inspect-usdz`, screenshot ve worklog gerekir.

1. Mental Model: Asset Journey



RealityKit asset pipeline

```

flowchart LR
    A[Gameplay ihtiyacı] --> B[Asset brief]
    B --> C["Mesh / origin / scale"]
    C --> D["UV + material + texture"]
    D --> E[USDZ export]
    E --> F[Assets/Imported]
    F --> G[asset_manifest.json]
    G --> H[xcodegen generate]
    H --> I[Xcode build]
    I --> J[RealityKit loader]

```

J --> K[Simulator screenshot]

K --> L[Worklog + checklist]

Kaynak şema: `Docs/diagrams/pipeline.mmd`, görüntülenebilir SVG: `Docs/diagrams/pipeline.svg`.

The Chain of Custody

Asset pipeline'ı bir teslim zinciri gibi düşünün:

Aşama	Çıktı	Kabul Kriteri
Gameplay ihtiyacı	Tek cümlelik asset amacı	Asset'in neden var olduğu açık
Asset brief	Ölçü, origin, bütçe, materyal beklentisi	Üretim başlamadan sınırlar belli
Mesh	Model geometri	Triangle bütçesine uyuyor
UV + material	Texture bağlanabilir yüzey	UV primvar ve material node zinciri net
USDZ export	<code>.usdz</code> dosyası	Texture embed veya paket ilişkisi doğrulanmış
Xcode resource	App bundle içeriği	Dosya <code>.app</code> içine giriyor
RealityKit loader	Runtime entity	HUD/sahne doğru asset'i gösteriyor
Screenshot	Görsel kanıt	Scale/origin/orientation/material okunuyor
Worklog	Öğrenme hafızası	Aynı hata tekrar yaşandığında çözüm bulunabiliyor

2. Project Map

Yol	Görev
<code>Assets/Imported</code>	App'e girecek <code>.usdz</code> assetleri
<code>Assets/Textures</code>	Ayrı tutulan texture kaynakları veya exportları
<code>Tools/asset_manifest.json</code>	Asset adı, bütçe, status ve notlar
<code>Sources/RealityKitPipelineDemo</code>	SwiftUI + RealityKit runtime kodu

Yol	Görev
Docs/WORKLOG.md	Sprint sonucu, karar ve doğrulama günlüğü
Docs/blender-usdz-checklist.md	Export sırasında kontrol listesi
Docs/asset-budget.md	Mobil mesh/texture bütçesi
Docs/diagrams	Guide ve PDF için şema kaynakları
Docs/screenshots	Public rehberde kullanılan seçilmiş simulator görsel kanıtları
Docs/pdf	Paylaşılabilir PDF çıktıları
Build	Lokal scratch build, DerivedData ve geçici screenshot çıktıları

3. Core Concepts

3.1 Scale

Tanım: Asset'in dünya içindeki fiziksel boyutu. Bu projede temel sözleşme `1 Blender unit = 1 meter`.

Neden önemli: RealityKit kamerasında küçük bir model dev gibi görünebilir. Blender'da doğru görünen boyut, oyun kamerasında test edilmeden kabul edilmez.

Bu projedeki ders: `target_basic.usdz` doğru import edildi ama ekranda çok büyüktü. RealityKit tarafında `0.48` uniform scale ile playable hale getirildi.

Kendini test et: Bir target'ı `0.3`, `0.48`, `0.75` scale ile açıp screenshot karşılaştırması yap. Hangisi oynanabilir alanı ve UI'ı daha iyi koruyor?

3.2 Origin / Pivot

Tanım: Entity'nin konumlandırma ve rotasyon merkezi.

Neden önemli: Target, gameplay pivot'u merkezde değilse spawn, rotation, collision ve hit detection beklenmedik davranır.

Kontrol: Asset sahneye geldiğinde pozisyonu değiştirince model beklenen merkezden hareket ediyor mu? Rotation uygulandığında merkez etrafında mı dönüyor?

3.3 UV

Tanım: 2D texture'ın 3D mesh üzerine nasıl sarılacağını belirleyen koordinatlar.

Neden önemli: Texture dosyası doğru olsa bile UV yanlışsa görsel ters, kaymış veya parçalı görünür.

Bu projedeki kritik ders: Blender USD export, aktif UV layer yerine shader'daki UV Map node'unun `uv_map` alanına bakar. Kaynak USDZ `st` primvar kullandığı için düzeltilmiş UV'leri `st` layer'ına yazmak gerekti.

Kabul kriteri: Texture rings hedefin merkezine oturuyor; ters, kaymış veya parçalanmış görünmüyor.

3.4 Material

Tanım: Mesh yüzeyinin shader ayarları: base color, roughness, metallic, texture bağlantıları.

İlk ders kuralı: Sadece base color texture kullan. Roughness ve metallic'i texture map değil material value olarak bırak.

Neden: İlk importta sorun çıktığında hata yüzeyini küçük tutar. Base color çalışmadan roughness/normal map eklemek debug maliyetini yükseltir.

3.5 Texture

Tanım: Material'ın görsel bilgisini taşıyan image map.

Bu projedeki başlangıç bütçesi: 512x512 PNG yeterli. 1024x1024 sadece simulator screenshot farkı açıkça gösterirse kullanılacak.

Kabul kriteri: Texture okunur, UV yönü doğru, dosya boyutu mobil bütçe içinde.

3.6 USDZ

Tanım: Apple platformlarında 3D model, material ve texture taşıyabilen paket formatı.

Kontrol sorusu: Texture USDZ içine gömülü mü, yoksa dış dosya path'ine mi bağlı kalmış?

Pratik not: `export_textures_mode='NEW'` bazı path uyarıları verebilir; asıl kabul RealityKit ve simulator screenshot sonucudur.

3.7 Xcode Resource Bundle

Tanım: App build edildiğinde resource dosyalarının `.app` bundle içine kopyalanması.

Bu projedeki yol: Asset `Assets/Imported` altına konur, sonra `xcodegen generate` veya `make generate` çalıştırılır.

3.8 RealityKit Loader Fallback

Tanım: Runtime önce gerçek asset'i arar; yoksa procedural placeholder ile çalışmaya devam eder.

Neden önemli: Asset pipeline bozulduğunda gameplay development durmaz. Sprint 3'te loader önce `target_basic_textured`, yoksa `target_basic` deniyor.

4. Sprint Walkthroughs

Sprint 1: First USDZ Import

Öğrenme hedefi: İlk gerçek target asset'ini app resource pipeline'a almak.

Yapılanlar:

- `target_basic.usdz` dosyası `Assets/Imported` altına eklendi.
- `Tools/asset_manifest.json` içinde status `imported` yapıldı.
- XcodeGen sonrası dosyanın app bundle'a girdiği doğrulandı.
- RealityKit loader asset'i yükledi; asset yoksa procedural fallback çalışmaya devam etti.

Görsel QA:

19:24



SCORE

40

SHOTS

9

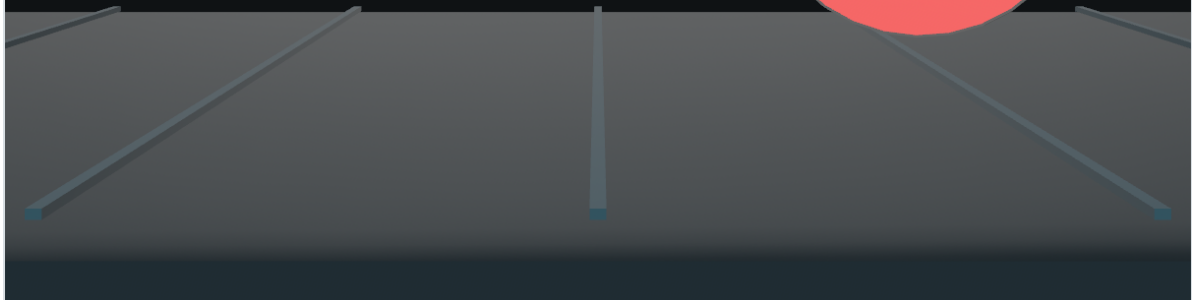
HITS

4

ACCURACY

44%

Imported target ready



Imported target front-facing

Bu görüntüde hedef artık kameraya dönük. İlk testte asset edge-on görünüyordu; nested mesh child rotation ve front-face düzeltmesiyle kırmızı/beyaz halkalar okunur hale geldi.

Öğrenme notu: İlk importta “dosya yüklendi” yeterli değil. Orientation ve child transform ayrı doğrulanmalı.

Acceptance: Asset bundle’da var, HUD imported target durumunu gösteriyor, screenshot’ta hedefin ön yüzü okunuyor.

Sprint 2: Scale and Spawn Tuning

Öğrenme hedefi: Target’ın playable görünmesini sağlamak.

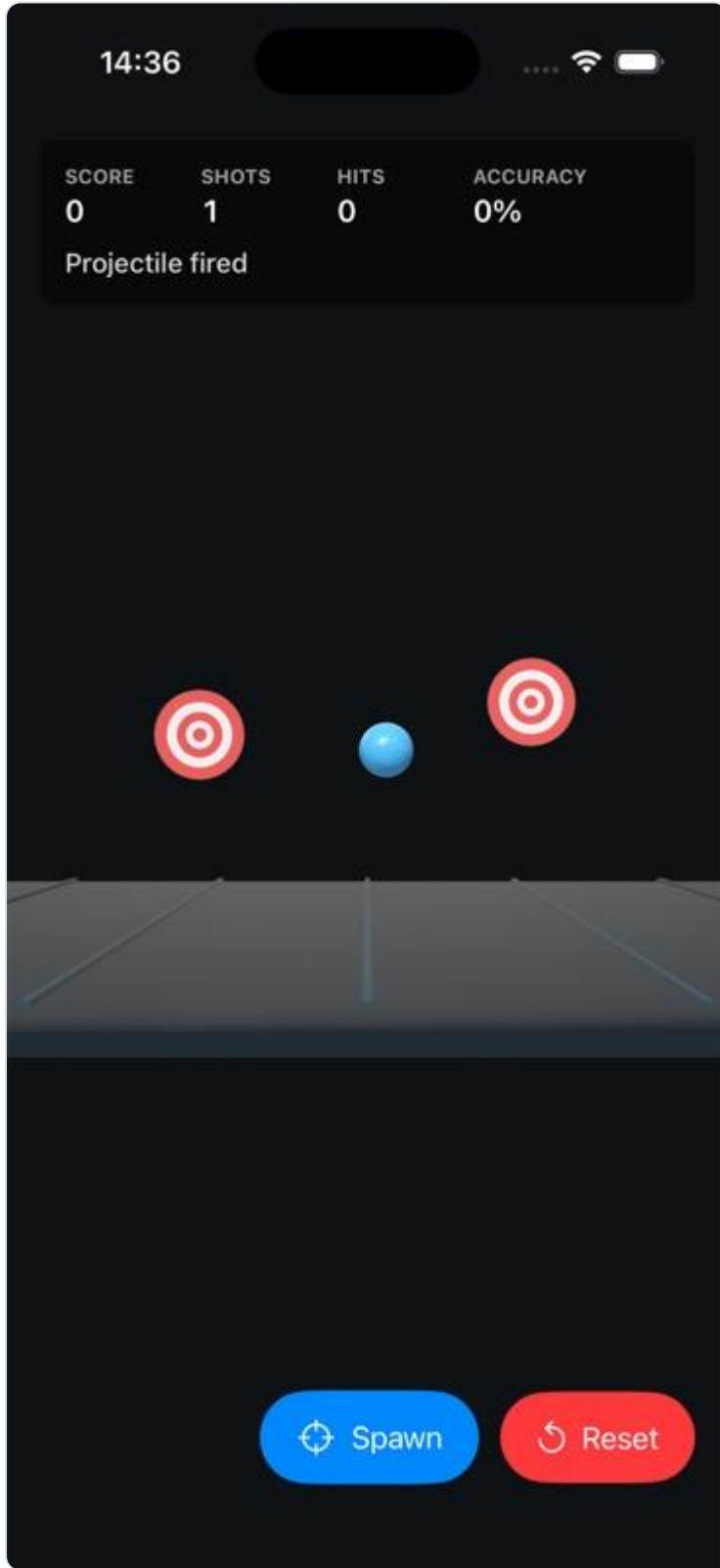
Problem:

- Asset doğru yöne bakıyordu ama çok büyüktü.
- Bazı spawn’lar kadraj dışına veya UI alanına yakına düşüyordu.

Çözüm:

- Imported target için `0.48` uniform scale uygulandı.
- Random spawn yerine sabit kadraj içi slotlar eklendi.
- Reset sonrası slot sırası sıfırlanarak deterministic test elde edildi.

Görsel QA:



Scaled target spawn slots

Bu görüntüde iki target kadraj içinde, floor referansına göre okunur ölçekte ve alt butonlarla çakışmadan görünüyor.

Öğrenme notu: Eğitim ve debugging sırasında deterministic sahne random sahneden daha değerlidir.

Acceptance: Target'lar HUD, floor ve control button'larla çakışmadan görünür; reset sonrası test sahnesi tekrar üretilebilir.

Sprint 3: First Textured Asset

Öğrenme hedefi: Tek base color texture içeren ilk USDZ asset'i RealityKit'e almak.

Yapılanlar:

- `target_basic_textured.usdz` üretildi.
- Kaynak geometri korundu.
- 512x512 PNG base color texture embed edildi.
- Tek `mat_textured` materyali kullanıldı.
- HUD'da `target_basic_textured ready` görüldü.

Görsel QA:

18:15



SCORE

SHOTS

HITS

ACCURACY

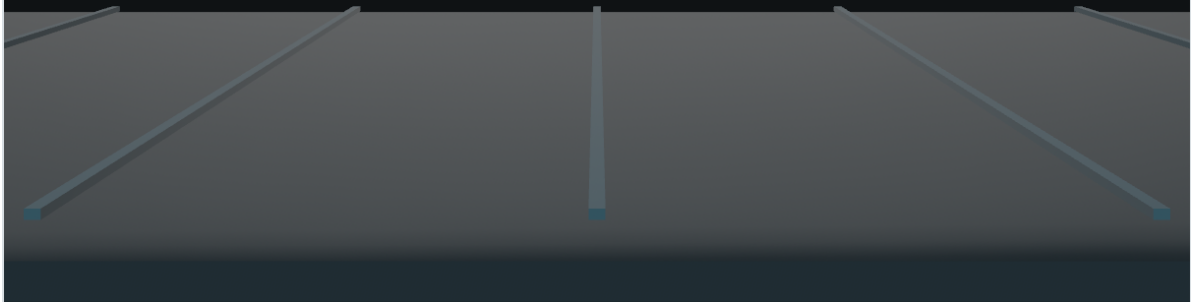
0

0

0

0%

target_basic_textured ready



Textured target loaded in RealityKit

Bu görüntüde texture'lı target RealityKit'te yüklenmiş durumda. Halkalar merkezde, UV yönü doğru ve scale/origin önceki target ile tutarlı.

Kritik ders: Blender USD export, shader'daki UV Map node'unun `uv_map` alanına bakar. Aktif UV layer tek başına yeterli değildir. Kaynak USDZ `st` primvar kullandığı için yeni UV'yi `st` layer'ına yazmak gerekti.

Acceptance: `Tools/asset_manifest.json` status `imported`; HUD `target_basic_textured ready`; screenshot'ta texture ters/kaymış değil.

5. Hands-on Labs

Lab A: Manifest Okuma

Amaç: Asset status ve bütçe bilgisini yorumlamak.

- `Tools/asset_manifest.json` dosyasını aç.
- `target_basic_textured` kaydını bul.
- `maxTriangles`, `maxTextureSize`, `textureMaps` ve `notes` alanlarını oku.
- Kendi cümlele bu asset'in neden kabul edildiğini yaz.

Başarı kriteri: Asset'in triangle, texture, material ve doğrulama bilgisini manifest üzerinden açıklayabiliyorsun.

Lab B: Loader Fallback Mantığı

Amaç: Asset yokken gameplay'in neden durmadığını anlamak.

- `GameARView.swift` içinde `loadTargetAsset()` fonksiyonunu oku.
- Loader sırasını not et.
- `target_basic_textured.usdz` yoksa ne olacağını sözlü açıkla.

Başarı kriteri: Fallback'in gameplay development hızını nasıl koruduğunu açıklayabiliyorsun.

Lab C: New Texture Variant

Amaç: Aynı pipeline'ı yeni bir görsel varyasyonda tekrar etmek.

1. Yeni asset id seç: `target_basic_blue_textured`.
2. 512x512 tek base color texture üret.
3. UV primvar adını doğrula.
4. USDZ export al.
5. Manifest kaydı ekle.
6. `rkp verify-asset target_basic_blue_textured --build` ile paket kapısını çalıştır.
7. Simulator screenshot ile görsel kabulü tamamla.

Başarı kriteri: Yeni texture varyasyonu manifest, build, screenshot ve worklog notuyla kapanıyor.

6. Debugging Playbook

Belirti	Muhtemel Sebep	Kontrol	Çözüm
Asset görünmüyor	Bundle'a girmedi veya path yanlış	<code>.app/Imported</code> içinde dosya var mı?	<code>make generate</code> , manifest ve resource path kontrolü
Procedural fallback görünüyor	Imported asset bulunamadı	HUD status ve loader sırası	Dosya adını asset id ile eşleştir
Asset edge-on görünüyor	Export axis veya child rotation farklı	Simulator screenshot	Entity veya child orientation düzelt
Asset çok büyük	Scale oyun kamerasına uygun değil	Screenshot ve floor referansı	RealityKit scale normalize et veya Blender scale düzelt
Texture ters/kaymış	UV projection veya primvar yanlış	UV Map node <code>uv_map</code> alanı	Doğru UV'yi <code>st</code> veya beklenen primvar'a yaz
Texture hiç görünmüyor	Texture embed edilmedi veya material bağlı değil	USDZ inspect / Reality Composer Pro	ImageTexture -> Base Color bağlantısını ve export mode'u kontrol et
Build lokal geçiyor ama simulator farklı	Eski bundle/app cache	HUD status ve screenshot timestamp	

Belirti	Muhtemel Sebep	Kontrol	Çözüm
			App'i yeniden build/run et, gerekirse simulator app'i sil
Hit detection garip	Collision shape asset ölçüsüne uymuyor	Projectile target mesafesi	Collision radius veya mesh bounds ayarla

7. Quality Rubric

Seviye	Tanım
0 - Not started	Asset yok veya manifest kaydı yok.
1 - File present	.usdz dosyası var ama build/runtime doğrulaması yok.
2 - Bundle verified	Build geçiyor ve asset bundle'a giriyor.
3 - Runtime verified	RealityKit sahnesinde doğru asset yükleniyor.
4 - Visual accepted	Screenshot scale/origin/orientation/material açısından kabul edildi.
5 - Teaching complete	Worklog, checklist, manifest ve screenshot güncel; ekip dersi açıklayabiliyor.

Bu projede public guide'a girecek assetler için hedef seviye **5**.

8. Asset Pipeline Learning Coverage and Roadmap

Bu rehber şu anda Sprint 1-3 kapsamını production seviyesinde anlatıyor: ilk USDZ import, scale/orientation düzeltmesi ve base color texture pipeline. Aşağıdaki modüller eğitim içeriğinin kalan yol haritasıdır.

Coverage Matrix

Modül	Durum	Kanıt	Not
	Complete		

Modül	Durum	Kanıt	Not
Project mental model		<code>Docs/guide.md</code> , pipeline diagram	Asset journey netleşti.
First USDZ import	Complete	<code>target_basic.usdz</code> , screenshot	Bundle + RealityKit import doğrulandı.
Scale/orientation tuning	Complete	<code>target_basic_scale_slots.jpg</code>	Playable framing çözüldü.
Base color texture	Complete	<code>target_basic_textured.usdz</code>	512x512 PNG embed doğrulandı.
UV primvar debugging	Complete	Worklog + checklist	<code>st</code> primvar dersi işlendi.
Manifest/worklog discipline	Complete	<code>Tools/asset_manifest.json</code> , <code>Docs/WORKLOG.md</code>	Handoff standardı oturdu.
CLI asset draft generation	Complete	<code>rkp make-asset</code> , <code>prompt-asset</code> , Meshy/Claude flags	Draft üretim yolları ayrıldı.
USDZ inspection gate	Complete	<code>rkp inspect-usdz</code> , tests	Texture presence, texture dimension, UV signal ve known triangle budget raporlanıyor.
Asset verification gate	Complete	<code>rkp verify-asset</code> , tests	Build, inspect, acceptance ve release kapıları tek akışta birleşti.
Roughness / metallic maps	Planned	yok	Material response dersi eklenecek.
Normal map	Planned	yok	Tangent-space normal ve export davranışı test edilecek.
Texture resolution comparison	Planned	yok	512 vs 1024 simulator/device karşılaştırması yapılacak.
Mobile performance	Planned	<code>Docs/asset-budget.md</code> başlangıç	Triangle, texture memory, material

Modül	Durum	Kanıt	Not
			slot, draw call anlatılacak.
Collision and gameplay asset fit	Started	ring-based score in <code>GameARView</code>	Visual texture gameplay scoring'e bağlandı; collision shape dersi genişletilecek.
Hit VFX / animation	Started	spark VFX, SDK-stable target spawn animation	Gameplay feedback başladı; ParticleEmitter/ audio açık.
Environment asset	Complete	<code>arena_floor.usdz</code> , <code>arena_floor_imported.jpg</code>	Floor scale/origin, target readability, and UV grid behavior verified.
Modern RealityKit feel	Started	physics bodies, collision events, PBR helper materials	ParticleEmitter/ audio/material comparison still open.
Device QA	Planned	simulator ağırlıklı	Gerçek cihaz frame time/thermal/touch kontrolü eklenecek.
Authoring workflow	Planned	kısmi	<code>.blend</code> kaynakları, export scripts, asset versioning kararı verilecek.

Planned Module 4: Texture Maps and Material Response

Amaç: Base color dışında material response kavramlarını öğretmek.

Öğrenilecekler:

- Roughness map ne zaman gerekir?
- Metallic map ne zaman gereksizdir?
- Normal map görsel kaliteyi nasıl artırır, maliyeti nedir?
- Tek material value ile texture map arasındaki fark.
- RealityKit imported material davranışı nasıl screenshot ile doğrulanır?

İlk egzersiz:

1. `target_basic_textured` için roughness value varyasyonu oluşturun.
2. Aynı asset'in roughness map kullanan varyasyonunu oluşturun.
3. Simulator screenshotlarını yan yana karşılaştırın.
4. Eğer fark okunmuyorsa map yerine material value kullanın.

Planned Module 5: Performance and Mobile Asset Budget

Amaç: Asset kararlarını mobil performansla ilişkilendirmek.

Öğrenilecekler:

- Triangle budget nasıl okunur?
- 512, 1024 ve 2048 texture memory farkı nedir?
- Material slot sayısı neden önemlidir?
- Draw call mental model'i.
- Instruments veya Xcode GPU capture ile başlangıç seviye frame kontrolü.

İlk egzersiz:

1. Aynı target'ı 512 ve 1024 base color texture ile export et.
2. Dosya boyutunu ve simulator görüntüsünü karşılaştır.
3. Fark yoksa 512'yi default kabul et.

Planned Module 6: Collision, VFX, and Gameplay Feel

Amaç: Görsel asset'in gameplay sistemleriyle ilişkisini öğretmek.

Öğrenilecekler:

- Visual mesh ve collision shape neden farklı olabilir?
- Collision sphere ne zaman yeterlidir?
- Asset scale collision radius'u nasıl etkiler?
- Hit VFX ve spawn animation oyuncuya ne anlatır?

İlk egzersiz:

1. Target collision radius'unu küçük/büyük varyasyonlarla test et.
2. Hit feedback için basit scale pulse veya color flash ekle.
3. Screenshot/video ile oyuncunun hit'i anlayıp anlamadığını değerlendir.

Planned Module 7: Environment Asset and Texture Atlas

Amaç: Tek hedef asset'inden environment pipeline'a geçmek.

Öğrenilecekler:

- `arena_floor.usdz` üretimi.
- Environment asset origin ve scale davranışı.
- Repeating texture ve atlas mantığı.
- Environment mesh ile gameplay target mesh bütçelerinin farkı.
- Floor asset'i target readability'yi bozuyor mu?
- Imported environment fallback: asset yoksa procedural floor çalışmaya devam etmeli.

İlk egzersiz:

1. Procedural floor yerine düşük poly `arena_floor.usdz` ekle.
2. Tek 512 texture veya atlas kullan.
3. Target contrast ve readability screenshot ile doğrula.
4. Manifest status'ünü `imported` yap ve öğrenme notunu worklog'a yaz.

Current project note:

- `arena_floor.usdz` is imported: 3.2m x 3.2m, centered origin, 128 triangles, `st` UV primvar, embedded 512x512 base color texture.
- Evidence: `Docs/screenshots/arena_floor_imported.jpg`.
- The imported floor grid is visible without reducing target readability.

Planned Module 8: Repo and Authoring Workflow

Amaç: Asset üretimini ekip çalışmasına uygun hale getirmek.

Öğrenilecekler:

- `.blend` dosyaları bu repo'da mı, ayrı art repo'da mı tutulmalı?
- Export script'leri nasıl versiyonlanmalı?
- Asset versioning manifest içinde nasıl izlenmeli?
- Release öncesi hangi dosyalar public repo'ya girmemeli?

İlk egzersiz:

1. Bir export script dosyasını `Tools` altına koy.

2. Aynı asset'i script ile tekrar üret.
3. Output hash, file size ve screenshot farkını kaydet.

Planned Module 9: Modern RealityKit Feel

Amaç: Asset pipeline demosunu modern RealityKit gameplay hissine yaklaştırmak.

Current project note:

- Projectile and target entities now use `PhysicsBodyComponent`.
- Projectile bodies are `.kinematic`, but their position is advanced manually in the game loop so they keep a flat aim line without gravity.
- Hit resolution listens to `CollisionEvents.Began`, with the previous distance check retained as fallback.
- Procedural showcase materials use a small `PhysicallyBasedMaterial` helper.
- Target spawn uses `move(to:relativeTo:duration:)` so the repo still builds on the iOS 18/Xcode 16 public CI baseline.

Ders: Apple docs'taki modern API'leri körlemesine eklemek doğru değil. Önce deployment target ve availability okunmalı; sonra yeni API ya guarded kullanılmalı ya da eski platformda fallback kalmalı.

9. Game Development Curriculum Roadmap

Bu bölüm, repo'nun asset pipeline eğitiminden tam oyun geliştirme eğitime dönüşmesi için gereken modülleri listeler. Bu modüller henüz tamamlanmış kabul edilmez.

Module 9: Game Loop Architecture

Amaç: Prototype'ı sadece sahne + target değil, açık state akışı olan bir mini oyuna çevirmek.

Öğrenilecekler:

- `idle`, `playing`, `waveCleared`, `gameOver`, `paused` state'leri.
- `GameSession`'in skor tutmaktan state yönetmeye genişlemesi.
- Reset, start, pause ve replay akışı.
- State değişimlerinin UI ve RealityKit sahnesine etkisi.

İlk egzersiz:

1. `GameState` enum ekle.
2. Start button olmadan projectile fire etmeyi engelle.
3. Wave clear olunca kısa bekleme ve sonraki wave state'i ekle.
4. State transition'ları worklog'a kaydet.

Module 10: Player Input and Game Feel

Amaç: Tap-to-shoot davranışını okunur, kontrollü ve öğretilebilir hale getirmek.

Öğrenilecekler:

- Crosshair veya aim indicator.
- Shot cadence / cooldown.
- Projectile speed ve travel time.
- Miss feedback.
- Input ile camera/ray arasındaki ilişki.

İlk egzersiz:

1. Tap cooldown ekle.
2. Crosshair görseli ekle.
3. Projectile speed için 3 farklı değer test et.
4. Hangi değer daha iyi hissettirdiğini screenshot veya kısa notla açıklayın.

Module 11: Waves, Difficulty, and Scoring

Amaç: Tekil hedef spawn'ından dengelenebilir oyun loop'una geçmek.

Öğrenilecekler:

- Wave başına target sayısı.
- Target health.
- Time limit.
- Accuracy ve miss penalty.
- Difficulty scaling.

İlk egzersiz:

1. Wave counter ekle.

2. Her wave'de target sayısını artır.
3. Miss penalty ekle.
4. Score formula'yı dokümente et.

Current project note:

- The target texture now has gameplay meaning: bullseye hits score `+5`, inner ring hits score `+3`, outer ring hits score `+1`.
- Hit scoring uses tap-time screen-space distance from deterministic target slot centers. This keeps the non-AR teaching prototype deterministic and directly ties visible rings to score.
- Evidence: `Docs/screenshots/ring_scoring_inner_hit.jpg` shows `Inner ring +3`, score `3`, hits `1`, accuracy `100%`.
- This is intentionally still lightweight. The next scoring lesson should add wave-level balancing and miss penalty.

Module 12: UI/UX Flow

Amaç: Prototype HUD'ını gerçek oyun akışına dönüştürmek.

Öğrenilecekler:

- Start screen.
- In-game HUD.
- Pause overlay.
- Results screen.
- Mini tutorial/hint.

İlk egzersiz:

1. Start screen ekle.
2. Results ekranında score, accuracy ve wave göster.
3. Restart akışını test et.

Module 13: Feedback Systems: VFX, Audio, Haptics

Amaç: Oyuncuya hit, miss, wave clear ve game over durumlarını hissettirmek.

Öğrenilecekler:

- Hit flash veya scale pulse.

- Spawn animation.
- Simple sound effects.
- Haptic feedback.
- Feedback'in gameplay readability'ye etkisi.

İlk egzersiz:

1. Target hit olduğunda kısa scale pulse veya color flash ekle.
2. Miss durumunda hafif UI feedback ver.
3. Feedback'in dikkat dağıtıp dağıtmadığını screenshot/not ile değerlendir.

Module 14: Persistence and Settings

Amaç: Tek oturumluk prototype'tan küçük ama gerçek app davranışına geçmek.

Öğrenilecekler:

- High score kaydı.
- Sound/haptic settings.
- Last selected mode.
- UserDefaults sınırları.

İlk egzersiz:

1. High score kaydet.
2. Reset sonrası high score'un kaldığını doğrula.
3. Sound/haptic toggle state'ini sakla.

Module 15: Performance Profiling on Device

Amaç: Simulator doğrulamasından gerçek cihaz performans disiplinine geçmek.

Öğrenilecekler:

- Frame time.
- Texture memory.
- Thermal/battery düşüncesi.
- Instruments veya Xcode GPU capture'a giriş.
- Asset budget kararlarını gerçek ölçümle bağlamak.

İlk egzersiz:

1. Aynı sahneyi simulator ve cihazda çalıştır.
2. Frame stutter veya thermal gözlemi yap.
3. Texture size değişikliğinin fark yaratıp yaratmadığını not et.

Module 16: Release and Collaboration Workflow

Amaç: Prototipi ekip içinde sürdürülebilir ve yayınlanabilir hale getirmek.

Öğrenilecekler:

- GitHub issues.
- PR review checklist.
- 1.
- Release notes.
- TestFlight/App Store yoluna giriş.

İlk egzersiz:

1. GitHub issue template ekle.
2. PR template ekle.
3. CI'da manifest parse ve build doğrulaması çalıştır.
4. `v0.1.0` release notes taslağı hazırla.

10. New Asset Checklist

Bu checklist yeni asset eklerken takip edilecek kısa reçetedir.

1. Asset ihtiyacını tek cümleyle yaz.
2. Asset id seç: `snake_case`.
3. Mesh ölçüsünü metre olarak belirle.
4. Origin/pivot kararını yaz.
5. Triangle ve texture bütçesini `Tools/asset_manifest.json` içinde belirle.
6. UV unwrap yap.
7. İlk denemede tek base color texture kullan.
8. `.usdz` export al.
9. Dosyayı `Assets/Imported` altına koy.

10. Manifest status ve notları güncelle.

11. `make generate` çalıştır.

12. Workspace-local build al:

```
make build
```

1. Simulator'da HUD ve görsel sonucu kontrol et.

2. Screenshot al.

3. `Docs/WORKLOG.md` ve ilgili checklist'e öğrenme notunu yaz.

10.5 Feature Planning Checklist

Asset dışındaki gameplay, UI, VFX veya sistem işleri için `Prompts/game-feature-brief.md` ile başlayın.

Minimum kabul standardı:

1. Player-facing değer tek cümleyle yazıldı.
2. Etkilenen game state veya runtime contract tanımlandı.
3. Asset varsa manifest ve fallback davranışı yazıldı.
4. Hit/miss/reset gibi edge case'ler belirtildi.
5. `make release-check` ve simulator kabul kriteri eklendi.
6. Worklog'a hangi dersin yazılacağı önceden biliniyor.

Bu standardın detaylı versiyonu `Docs/production-playbook.md` içindedir.

11. Glossary

Terim	Kısa açıklama
Asset	Oyunda kullanılan model, texture, ses veya benzeri üretim çıktısı.
USDZ	Apple ekosisteminde yaygın kullanılan paketlenmiş 3D asset formatı.
Primvar	USD içinde mesh üzerinde taşınan vertex/face-varying veri; UV için <code>st</code> yaygındır.

Terim	Kısa açıklama
UV	2D texture koordinatları.
Base color	Material'ın temel renk/texture girdisi.
Roughness	Yüzeyin ışığı ne kadar yaydığını belirleyen material değeri.
Metallic	Yüzeyin metal davranışı gösterip göstermediğini belirleyen material değeri.
Origin / pivot	Entity'nin transform merkezi.
Bundle	Build sonrası app içine kopyalanan resource paketi.
Fallback	Asıl asset yokken app'in procedural veya yedek asset ile çalışmaya devam etmesi.
Deterministic spawn	Aynı koşulda aynı sahnenin tekrar oluşması.

12. PDF / Repo Release Checklist

Repo public hale gelmeden önce:

- README `Docs/guide.md` dosyasına link veriyor.
- `Docs/guide.md` son sprintleri içeriyor.
- `Docs/diagrams/pipeline.svg` görüntülenebiliyor.
- `Tools/asset_manifest.json` parse oluyor ve status'lar doğru.
- Seçilmiş screenshot'lar gerçekten var.
- Büyük geçici build çıktıları public repo'ya girmiyor.
- PDF gerekiyorsa `Docs/pdf/realitykit-pipeline-guide.pdf` taze üretilmiş.

PDF üretmek için:

```
make guide
```

13. Appendix

Current Teaching Assets

Asset	Status	Ders
<code>target_basic.usdz</code>	imported	İlk USDZ import, orientation, scale
<code>target_basic_textured.usdz</code>	imported	Base color texture, UV primvar, embed
<code>arena_floor.usdz</code>	imported	Environment scale/origin, floor readability, UV grid behavior

Core Commands

```
rkp status
rkp make-asset enemy_drone --type gameplay_target --prompt "red bullseye drone target"
rkp verify-asset enemy_drone --build
rkp inspect-usdz enemy_drone --json
make release-check
```

Evidence Files

Dosya	Ne kanıtlıyor?
<code>Docs/screenshots/target_basic_frontface.png</code>	Imported target front-facing düzeltmesi
<code>Docs/screenshots/target_basic_scale_slots.jpg</code>	Scale ve deterministic spawn düzeltmesi
<code>Docs/screenshots/target_textured_sprint3_fresh.png</code>	Texture'lı asset RealityKit'te yüklendi
<code>Docs/screenshots/ring_scoring_inner_hit.jpg</code>	Texture ring'i gameplay skoruna bağlandı
<code>Docs/screenshots/arena_floor_fallback_ready.jpg</code>	<code>arena_floor.usdz</code> yokken procedural floor fallback çalışıyor
<code>Docs/screenshots/arena_floor_imported.jpg</code>	Imported arena floor target readability'yi bozmadan görünüyor

Instructor Notes

- İlk eğitim oturumunda amaç "asset yapmak" değil, zinciri anlamaktır.
- Hata çıktığında hemen fix yazmayın; önce hangi aşamada kırıldığını işaretleyin.
- Screenshot yorumunu birlikte yapın. Görsel QA öğrenmenin ana parçası.
- Yeni map veya texture türü eklemekten önce base color akışını tekrar ettirin.